

Day 9

Managing Users and Permissions in Linux

Student Notes: Every command explained in simple language

What you will learn	Simple meaning
Users and groups	Who can use the computer, and how to organise them into teams
Commands	useradd, passwd, groupadd, usermod, userdel, su, sudo and more
Important files	Where Linux stores user, password and group information
Practice	Step-by-step labs you can try on your own Linux machine

Tip: Practice these commands only on a practice machine or virtual machine (VM). Commands with **sudo** change the system, so read before you press Enter.

1. Overview: Why User Management?

Linux is a **multi-user** system. This means many people can use one computer at the same time, each with their own account. User and permission management means deciding **who** can log in and **what** each person is allowed to do. This keeps the system safe.

Simple terms you must know

Term	Meaning in simple words
User	A person (or program) that has an account on the system.
Group	A team of users. Give permission to the team once instead of to each person.
Root	The boss account. It can do everything, including deleting the whole system.
Permission	The rule that says who can read, write or run a file.
Home directory	A personal folder for each user, usually /home/username.
Shell	The program that reads the commands you type. /bin/bash is the most common.
sudo	A word placed before a command to run it with the boss (root) power.
Syntax	The correct way to write a command, like grammar for commands.
Option / Flag	A small extra added with a dash (for example -m) to change how a command works.
UID / GID	UID is the user's ID number. GID is the group's ID number.

Reading syntax: How to read syntax: In command [options] username, the words in [] are optional. Words like *username* are placeholders. Replace them with real values.

2. Types of Users

Type	What it is	Example
Root user	The administrator with full access to everything. UID is always 0.	root
System user	Accounts used by programs and services in the background. Nobody logs in with them.	daemon, www-data
Regular user	Normal person with limited permissions. UID usually starts from 1000.	sourabh

Command: whoami

Use: Shows the name of the user you are logged in as right now.

Syntax: whoami (no options needed)

```
whoami
```

Example output:
student

Command: id

Use: Shows the user's ID (uid), main group ID (gid) and all groups. Use it to check if a user exists.

Syntax: id [username] (if no username is given, it shows your own)

```
id root
```

Example output:
uid=0(root) gid=0(root) groups=0(root)

Remember: uid=0 proves that root is the administrator. Every other user has a bigger number.

Command: cat /etc/passwd

Use: cat means 'show the file on screen'. The file /etc/passwd holds the list of all users.

Syntax: cat filename

```
cat /etc/passwd
```

Finding system users

System users usually do not have a folder inside /home. So we can **filter** (search) the list. grep searches text and -v means 'show lines that do NOT contain this word'.

```
grep -v "/home" /etc/passwd
```

```
# Same idea, as written in your notes:  
less /etc/passwd | grep -v "/home"
```

The **pipe symbol |** takes the output of the first command and gives it to the second command. less is a page viewer. In practice, the first (simpler) form is enough.

3. Creating Users: useradd

Use: Creates a new user account.

Syntax: sudo useradd [options] username

Option	Meaning in simple words
-m	make a home directory (personal folder) for the user.
-s	Choose the shell (command-reading program) for the user, e.g. /bin/bash.

Option	Meaning in simple words
-G	Add the user to extra G roups (supplementary groups). Separate many groups with commas.

```
sudo useradd -m -s /bin/bash sourabh
```

Meaning, word by word:

Part	What it does
sudo	Run with administrator power (only root can create users).
useradd	The command to add a user.
-m	Create /home/sourabh.
-s /bin/bash	Give the user the bash shell.
sourabh	The new user's name.

Verify the user was created:

```
id sourabh
```

Example output:

```
uid=1001(sourabh) gid=1001(sourabh) groups=1001(sourabh)
```

Common mistake: If you forget **-m**, the user is created but has no home folder. Also, a new user has no password yet, so set one in the next step.

4. Setting Passwords: passwd

Use: Sets or changes the password of a user. While typing, nothing appears on screen. This is normal and is done for security.

Syntax: `sudo passwd username` (without sudo and username, you change your own password)

```
sudo passwd sourabh
```

```
New password: *****
```

```
Retype new password: *****
```

Practice: Test it: log in as the new user using the new password (see **su** in section 8).

5. Managing Groups

A **group** is a team of users. If a folder is given to the group *developers*, every member of that group can use it. This is easier than setting permissions for each person.

Command	Use	Syntax
groupadd	Create a new group	sudo groupadd groupname
usermod -aG	Add a user to a group	sudo usermod -aG groupname username
groups	Show groups of a user	groups [username]

Step 1: Create a group

```
sudo groupadd developers
```

Step 2: Add a user to the group

```
sudo usermod -aG developers sourabh
```

Part	What it does
usermod	M odify (change) an existing user.
-a	A ppend. Add to the group and keep the old groups.
-G	The G roup list to use.
developers	Name of the group.
sourabh	Name of the user.

Very important: Always use **-a** together with **-G**. If you write only **-G**, the user is removed from all other groups and kept only in the new one.

Step 3: Check group membership

```
groups sourabh
```

Example output:
sourabh : sourabh developers

6. Removing Users: userdel

Use: Deletes a user account.

Syntax: sudo userdel [options] username

Command	What happens
sudo userdel sourabh	Deletes the user account. The home folder /home/sourabh is kept .
sudo userdel -r sourabh	Deletes the account and the home folder with all its files (-r = remove).

Warning: Deleting with `-r` removes the user's files forever. Take a backup first if the files are important.

7. Important Files Used in User Management

File	What it stores
<code>/etc/passwd</code>	Basic account details: name, ID, home folder, shell. Everyone can read it.
<code>/etc/shadow</code>	Passwords in secret (encrypted) form. Only root can read it.
<code>/etc/group</code>	Group names and which users belong to each group.

Each line of `/etc/passwd` has 7 parts separated by a colon (:). Example line:

```
sourabh:x:1001:1001::/home/sourabh:/bin/bash
```

Part	Example	Meaning
1	sourabh	Username
2	x	Password is stored in <code>/etc/shadow</code>
3	1001	UID (user ID number)
4	1001	GID (main group ID number)
5	(empty)	Comment or full name (optional)
6	<code>/home/sourabh</code>	Home directory
7	<code>/bin/bash</code>	Login shell

Practice commands:

```
cat /etc/passwd      # list of users
cat /etc/group        # list of groups
sudo cat /etc/shadow  # needs sudo, passwords are encrypted
grep sourabh /etc/passwd # show only one user's line
```

A line in `/etc/group` looks like: `developers:x:1002:sourabh`. It means: group name : password placeholder : group ID : members.

8. User Home Directories

Each regular user has a personal folder, usually `/home/username`, for documents, downloads and settings. Other users cannot normally open it.

```
ls /home              # show all home folders
sudo ls /home/sourabh # look inside another user's home (root power)
```

ls means 'list': it shows the files and folders inside a directory. Syntax: `ls [options] [path]`

Fact: The root user's home is `/root`, not `/home/root`.

9. Switching Between Users

The su command (switch user)

Use: Lets you become another user in the same terminal. You need that user's password (root can switch without it).

Syntax: su [-] [username]

Command	Meaning
su sourabh	Become sourabh, but keep your old folder and settings.
su - sourabh	Become sourabh with a full fresh login (his home folder and settings). Recommended.
su	With no name, you try to become root.
exit	Leave the new user and go back to the previous user.

```
whoami      # student
su sourabh  # enter sourabh's password
whoami      # sourabh
exit        # back to student
```

The sudo command (superuser do)

Use: Runs **one** command with the power of root (or another user). After the command finishes, you are normal again. It asks for **your own** password, and only users allowed in the sudo list can use it.

Syntax: sudo [-u username] command

```
sudo apt update      # refresh the software list (Ubuntu/Debian)
sudo -u sourabh whoami # run a command as user sourabh
```

Point	su	sudo
Password asked	Target user's password	Your own password
Power lasts	Until you type exit	Only for that one command
Safety	Less safe	Safer, and actions are logged

10. Quick Revision Cheat Sheet

Command	What it does
whoami	Show current user
id username	Show UID, GID and groups
cat /etc/passwd	List all users

Command	What it does
<code>cat /etc/group</code>	List all groups
<code>sudo useradd -m -s /bin/bash name</code>	Create user with home folder and bash shell
<code>sudo passwd name</code>	Set or change password
<code>sudo groupadd team</code>	Create group
<code>sudo usermod -aG team name</code>	Add user to group
<code>groups name</code>	Show user's groups
<code>sudo userdel name</code>	Delete user (keep home)
<code>sudo userdel -r name</code>	Delete user and home folder
<code>ls /home</code>	List home folders
<code>su - name</code>	Switch to another user
<code>exit</code>	Return to previous user
<code>sudo command</code>	Run a command as root

11. Full Practice Lab (Do it in order)

1. `whoami` (check who you are)
2. `sudo useradd -m -s /bin/bash sourabh` (create user)
3. `id sourabh` (confirm user exists)
4. `sudo passwd sourabh` (set password)
5. `sudo groupadd developers` (create group)
6. `sudo usermod -aG developers sourabh` (add to group)
7. `groups sourabh` (verify membership)
8. `grep sourabh /etc/passwd` (see the file entry)
9. `sudo ls /home/sourabh` (see his home folder)
10. `su - sourabh`, then `whoami`, then `exit` (switch and return)
11. `sudo userdel -r sourabh` (clean up: delete user and home)

Practice questions

- What is the difference between **userdel** and **userdel -r**?
- Why should we use **-a** with **-G** in `usermod`?
- Which file stores the encrypted passwords?
- What is the difference between **su** and **sudo**?
- Name the three types of users in Linux.

Summary: Conclusion: Good user and permission management keeps a Linux system organised and secure. Practice daily to become confident.